Local AI Dev Assistant

Test Doc

1. Project Overview

1.1 What We're Building

A fully offline, air-gapped AI developer assistant. Developers ask questions about their codebase in their IDE and get answers grounded in actual project code — no internet, no third-party services, no data leaving the network.

The system uses Ollama to run models locally, a custom RAG service to index and search codebases, and Continue.dev as the IDE interface. It works in VS Code today and will support JetBrains in a future Step.

1.2 Environments

	Home Test Machine	Production
OS	Windows (internet-connected)	Ubuntu 24.04 LTS (air-gapped)
GPU	RTX 4060 8GB	Tesla P4 8GB GDDR5 (Pascal)
Driver / CUDA	Windows built-in	nvidia-driver-580-server + CUDA 12.x
Chat Model	llama3.1:8b — 48 tok/sec	gemma2:2b — 41 tok/sec
Embed Model	nomic-embed-text (port 11435)	nomic-embed-text (port 11435)
GPU Acceleration	**Confirmed**	**Confirmed**

Target model size: gemma2:2b for Step 1. Eventual target 7-8B once validated on production hardware.

2. Compliance & Model Approval

All models must be US-origin or explicitly approved. The primary approval basis is Meta's Llama family, covered under GSA OneGov procurement. Microsoft Phi models are acceptable as a secondary option. Chinese-origin models (Qwen/Alibaba, DeepSeek, Baidu) are excluded regardless of performance.

Model	Status	Reason
Meta Llama (all sizes)	**Approved**	US-origin, GSA OneGov approved, open weights, auditable
Microsoft Phi-4 / Phi-4-mini	**Approved**	US-origin (Microsoft), strong coding performance at small sizes
Google Gemma	**Approved**	US-origin (Google), currently in use — gemma2:2b on production
Qwen, DeepSeek, Baidu	**Excluded**	Chinese-origin — not approved regardless of performance

3. System Architecture

3.1 The Two Paths

Path A (active — Steps 1-4): Developer types in Continue → RAG context injected → Ollama generates answer → Continue displays. Continue is the display layer only, not an orchestrator.

Path B (future — Step 5+): Agent CLI → Python loop → Ollama → tools (read/write/run) → answer. This runs separately from Continue — agent frameworks do not integrate with Continue.dev directly.

3.2 Component Layers

Layer	Technology	Role
Inference	Ollama (two instances)	Port 11434 (chat), port 11435 (embeddings)
RAG Service	FastAPI + ChromaDB	Indexes code, serves context to IDE on port 8001
IDE Layer	Continue.dev (VS Code)	Developer interface — display only, not an orchestrator
Agent Layer	TBD (future Step)	Agentic tool use — separate from Continue

4. Project Steps

Active

- Step 1 — Inference Server: Two Ollama instances running, GPU acceleration confirmed, models loaded
- Step 2 — RAG Service: FastAPI + ChromaDB + tree-sitter indexer, /context endpoint returning real chunks
- Step 3 — Continue Integration: @rag wired to RAG service, model answering with codebase context — COMPLETE

Future

- Step 4 — Hardening: health checks, logging, incremental indexing, multi-repo support, team onboarding docs, containerized
- Step 5 — Agent Layer: agentic tool use (Aider / SmolAgents / CrewAI) — runs separately from Continue
- Step 6 — Production Scaling: multi-GPU, round-robin inference, Podman/Kubernetes orchestration

5. Tool Decisions

Category	Chosen	Rejected	Why
Inference	Ollama	llama.cpp direct, vLLM	Easiest setup, good GPU support, active project
Vector DB	ChromaDB	Qdrant, pgvector, FAISS	File-based, no server, simple Python API
Code Chunking	tree-sitter	Sliding window only	Keeps functions/classes intact, 17 languages
IDE Layer	Continue.dev	Tabby, Cody, Copilot	Open source, Ollama-native, custom HTTP context providers

6. Working System — Steps 1-3 Complete

6.1 What Was Built

- Ollama running llama3.1:8b on port 11434 — 48 tok/sec (RTX 4060)
- Second Ollama instance running nomic-embed-text on port 11435 (required — see 6.3)
- FastAPI RAG service on port 8001 with ChromaDB, tree-sitter chunking, sliding window fallback
- Continue.dev wired to Ollama + RAG service via single /context endpoint
- @rag returns real code chunks with file names and line numbers
- Model answers using actual codebase context — validated

6.2 Startup Procedure

Run these three in order every session:

```
# Terminal 1 — Ollama chat (already running as service on Linux)
ollama serve   # port 11434, handles chat requests

# Terminal 2 — Second Ollama instance (embeddings only)
$env:OLLAMA_HOST = '127.0.0.1:11435'   # Windows
$env:OLLAMA_MODELS = 'C:Users<user>.ollamamodels'
ollama serve   # port 11435, handles embed requests

# Terminal 3 — RAG service
cd rag-service && .venvScriptsActivate.ps1
uvicorn api:app --host 0.0.0.0 --port 8001
```

6.3 Why Two Ollama Instances

Ollama processes requests serially. When a developer uses @rag, Continue simultaneously fires a chat request (port 11434) and calls the /context endpoint which embeds the query (needs port 11435). On a single instance these collide — the embedding returns empty and RAG silently fails. Two instances, two queues. Both models fit in 8GB VRAM: llama3.1:8b uses ~5GB, nomic-embed-text uses ~300MB.

6.4 End-to-End Request Flow

- Developer types: @rag where is the add function?
- Continue POSTs to /context with fullInput (not query — query is always empty)
- workspacePath comes URL-encoded as file:///z%3A/... — endpoint decodes it
- Falls back to DEFAULT_REPO env var since Continue sends workspace root, not repo subfolder
- Embeds query via nomic-embed-text on port 11435
- Searches ChromaDB, returns top 4 chunks as [{name, description, content}]
- Continue shows '4 context items' and injects chunks into prompt
- Ollama (port 11434) generates response using real code context

6.5 Validated Test Results

Test	Result	Notes
llama3.1:8b loads with GPU	**PASS**	48 tok/sec, RTX 4060
nomic-embed-text on port 11435	**PASS**	274MB, ~300MB VRAM
Repo indexed with tree-sitter	**PASS**	22 chunks, minimal repo
/context returns code chunks	**PASS**	Direct Python test confirmed
@rag codebase context in Continue	**PASS**	Single /context endpoint workaround
Model cites correct file + line numbers	**PASS**	add() at minimal.py:23-25

7. Implementation Reference

Everything needed to set up or reproduce the system. Dev setup (Windows) and production setup (Ubuntu) are covered separately.

7.1 Environment File (rag-service/.env)

```
OLLAMA_HOST=http://localhost:11434
OLLAMA_EMBED_HOST=http://localhost:11435
OLLAMA_EMBED_MODEL=nomic-embed-text
CHROMA_PATH=./chroma
DEFAULT_REPO=Z:New-Dev-AI-Projectminimal
```

DEFAULT_REPO must match the exact path used when running indexer.py. Continue sends the workspace root via workspacePath, not the repo subfolder — DEFAULT_REPO overrides it.

7.2 Continue Config (~/.continue/config.yaml)

```
models:
  - name: Llama 3.1 8B
    provider: ollama
    model: llama3.1:8b
    apiBase: http://<server-ip>:11434
    contextLength: 8192

context:
  - provider: file
```

```
  - provider: http
    params:
      url: http://<server-ip>:8001/context
      title: rag
      displayTitle: Codebase RAG
```

Notes: use context: not contextProviders: (YAML schema change). One HTTP provider only — multiple HTTP providers in YAML is a confirmed Continue bug where only the first shows up.

7.3 Test /context Before Using Continue

```
python -c "import requests; r = requests.post(
  'http://localhost:8001/context',
  json={'query':'','fullInput':'where is add defined',
        'workspacePath':'Z:\\your\\repo'});
print(r.status_code, r.text)"
```

Working response: [{name: Codebase RAG, description: file.py:23-25, content: ...}]. Empty array [] means DEFAULT_REPO doesn't match an indexed collection — re-index or fix the path. 500 error means port 11435 is down.

7.4 RAG Service Files

File	What it does
indexer.py	Walks repo, chunks with tree-sitter (semantic) or sliding window (fallback), embeds via nomic, stores in ChromaDB. Run: python indexer.py --repo /path/to/repo
api.py	FastAPI app. POST /context endpoint — decodes workspacePath, uses fullInput as query, returns [{name, description, content}] array to Continue
ollama_embed.py	Sends embed requests to OLLAMA_EMBED_HOST (port 11435). Never touches port 11434.
repo_map.py	Builds plain-text map of repo — files, classes, functions, line numbers. Used inside /context response.
chroma/	ChromaDB persistent storage. One collection per repo. Back this up if you want to preserve indexed data.

tree-sitter semantic chunking supports: .py .js .jsx .ts .tsx .go .java .rs .c .h .cpp .cs .rb .php .lua. All other file types use sliding window (500 char window, 100 char overlap).

7.5 Dev Setup (Windows)

- Install Ollama from ollama.com — pulls models to C:Users<user>.ollamamodels
- ollama pull llama3.1:8b and ollama pull nomic-embed-text
- Install Continue extension in VS Code from marketplace (or .vsix for offline)
- Edit ~/.continue/config.yaml with config from 7.2
- cd rag-service && python -m venv .venv && .venvScriptsActivate.ps1
- pip install -r requirements.txt OR pip install fastapi uvicorn chromadb tree-sitter tree-sitter-languages requests python-dotenv
- Create .env from 7.1, set DEFAULT_REPO to your repo path
- python indexer.py --repo <your-repo-path>
- uvicorn api:app --host 0.0.0.0 --port 8001
- Test /context using command from 7.3, then open Continue and try @rag

7.6 Production Setup (Ubuntu 24.04, Air-Gapped)

What to Transfer (from internet-connected machine)

- Ollama binary — download from ollama.com/download, single binary ~50MB
- Ollama models — already transferred and verified
- Mirrored apt packages — linux-hwe kernel, nvidia-driver-580-server, cuda-toolkit
- rag-service/ folder — all Python files
- Python pip wheels — run pip download -r requirements.txt on internet machine, transfer the .whl files
- Continue .vsix extension file — download from VS Code marketplace or GitHub releases
- chroma/ folder — optional. Can transfer pre-indexed data or re-index on prod (recommended — Windows paths in collection metadata won't match Linux paths anyway)

1. Kernel & GPU Driver

- Install HWE kernel (better hardware support): sudo apt install linux-generic-hwe-24.04
- Reboot into new kernel: sudo reboot
- List available GPU drivers: sudo ubuntu-drivers list --gpgpu
- Should show nvidia-driver-580-server as an option for the Tesla P4
- Install driver: sudo apt install nvidia-driver-580-server
- Reboot: sudo reboot
- Verify: nvidia-smi — should show Tesla P4, driver version 580.x

2. CUDA Toolkit

- Install from mirrored packages: sudo apt install cuda-toolkit-12-x
- Verify: nvcc --version — should show CUDA 12.x

3. Ollama

- Copy ollama binary to server, make executable: chmod +x ollama
- Move to PATH: sudo mv ollama /usr/local/bin/ollama
- Models already transferred — verify: ollama list
- Create systemd service for port 11434 (chat): set OLLAMA_HOST=0.0.0.0 in service override
- Start second instance for embeddings: OLLAMA_HOST=127.0.0.1:11435 OLLAMA_MODELS=/path/to/models ollama serve &
- Verify GPU acceleration: start a model, run ollama ps — GPU % should be > 0

4. RAG Service

- Copy rag-service/ folder to server
- Install Python 3.11+: sudo apt install python3 python3-venv python3-pip
- Create venv: python3 -m venv .venv && source .venv/bin/activate
- Install from transferred wheels: pip install --no-index --find-links=./wheels -r requirements.txt
- Create .env — update paths to Linux format (e.g. /home/user/repos/myrepo)
- Do NOT transfer chroma/ folder — re-index on prod: python indexer.py --repo /path/to/repo
- Start: uvicorn api:app --host 0.0.0.0 --port 8001
- Test with Python requests command from section 7.3

5. Continue Extension (VS Code)

- Transfer .vsix file to developer machines
- Install offline: code --install-extension continue-*.vsix
- Edit ~/.continue/config.yaml — point apiBase and context url to server IP

8. Baseline QA & Model Evaluation

8.1 Testing Philosophy

Test the full pipeline — not just the model in isolation. A response is only as good as the context it receives. Every test should verify that RAG context items appear before evaluating the answer quality.

8.2 Test Scenarios

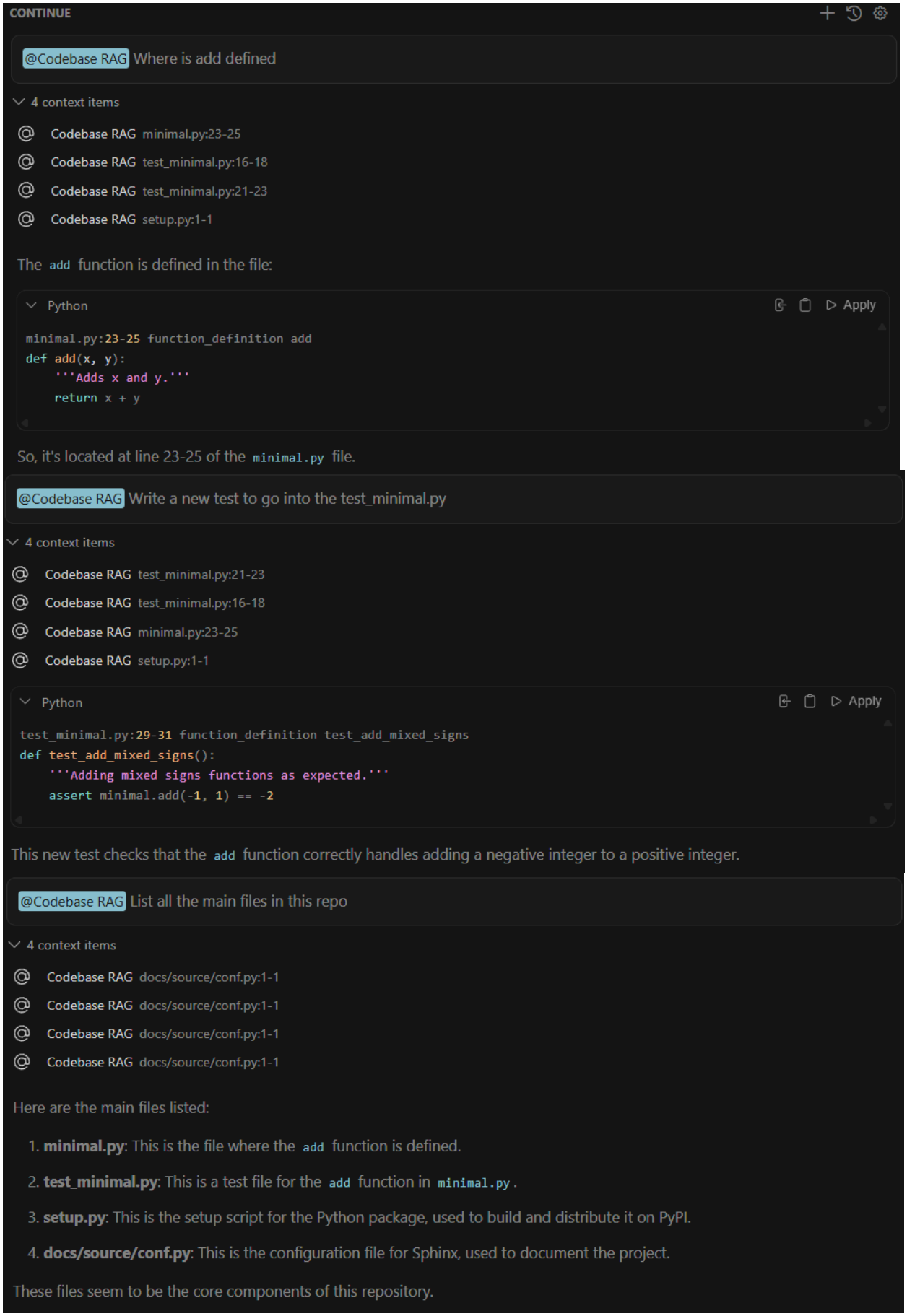
- Codebase orientation: @rag what does this project do? — expect summary using actual files
- Function location: @rag where is [function] defined? — expect correct file and line number
- Code generation: @rag add error handling to [function] — expect code that fits the actual codebase style
- RAG validation: ask about code not in any open file — if correct, RAG is working
- Performance: measure tokens/sec under load, check VRAM usage stays stable

8.3 Test Scenarios

The following is what the project looks like while working:

```
(.venv) PS Z:\New-Dev-AI-Project\rag-service> uvicorn api:app --host 0.0.0.0 --port 8001
--reload
Could not find platform independent libraries <prefix>
INFO:     Will watch for changes in these directories: ['Z:\\New-Dev-AI-Project\\rag-service']
INFO:     Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [15088] using WatchFiles
Could not find platform independent libraries <prefix>
INFO:     Started server process [17060]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:62252 - "POST /context HTTP/1.1" 200 OK
```

Continue takes in the user prompt and since the @rag context is in the continue configuration file it will use the URL `"http://localhost:8001/context"` as the context provider endpoint. The request is made to the API server and the top 4 chunks are returned as context to continue. The model with then proceed to answer with the given context. Below are a few test result screenshots.


CONTINUE
@Codebase RAG Where is add defined
4 context items
Codebase RAG minimal.py:23-25
Codebase RAG test_minimal.py:16-18
Codebase RAG test_minimal.py:21-23
Codebase RAG setup.py:1-1
The add function is defined in the file:
Python
Apply
minimal.py:23-25 function_definition add
def add(x, y):
'''Adds x and y.'''
return x + y
So, it's located at line 23-25 of the minimal.py file.
@Codebase RAG Write a new test to go into the test_minimal.py
4 context items
Codebase RAG test_minimal.py:21-23
Codebase RAG test_minimal.py:16-18
Codebase RAG minimal.py:23-25
Codebase RAG setup.py:1-1
Python
Apply
test_minimal.py:29-31 function_definition test_add_mixed_signs
def test_add_mixed_signs():
'''Adding mixed signs functions as expected.'''
assert minimal.add(-1, 1) == -2
This new test checks that the add function correctly handles adding a negative integer to a positive integer.
@Codebase RAG List all the main files in this repo
4 context items
Codebase RAG docs/source/conf.py:1-1
Codebase RAG docs/source/conf.py:1-1
Codebase RAG docs/source/conf.py:1-1
Codebase RAG docs/source/conf.py:1-1
Here are the main files listed:
1. minimal.py: This is the file where the add function is defined.
2. test_minimal.py: This is a test file for the add function in minimal.py.
3. setup.py: This is the setup script for the Python package, used to build and distribute it on PyPI.
4. docs/source/conf.py: This is the configuration file for Sphinx, used to document the project.
These files seem to be the core components of this repository.

8.4 Test Scenarios

Environment	Model	Tokens/sec	Notes
RTX 4060 (Windows dev)	llama3.1:8b Q4	48	GPU acceleration confirmed
Tesla P4 (Ubuntu prod)	gemma2:2b	41	GPU acceleration confirmed

— Future Steps —

The sections below are reference material for future work. Nothing here is active.

---

9. Agent Frameworks (Future)

An agent layer adds tool use — the model can read files, run commands, and write code autonomously rather than just answering questions. This runs as a separate process from Continue.dev, not integrated with it.

Options being evaluated: Aider (VS Code extension, plan-review-run workflow, safest execution model), SmolAgents (lightweight Python loop), CrewAI (multi-agent coordination). Decision TBD until Step 3 is fully hardened.

Framework	Type	Notes
Aider	CLI/VS Code	Plan-review-run workflow. Developer approves each step. Safest. Install offline via VSIX.
SmolAgents	Python library	Lightweight, minimal dependencies. Good for simple tool loops.
CrewAI	Python library	Multi-agent coordination. More complex, better for multi-step workflows.

---

10. Containerization (Future)

Podman is preferred over Docker for government environments — rootless by default, no daemon, SELinux compatible. When containerizing: one container for Ollama, one for the RAG service. Podman Compose or Kubernetes for orchestration. For now, systemd services on Ubuntu are sufficient.

---

11. Ecosystem & Design Alternatives

11.1 Comparable Projects

Project	Notes
Tabby	Open source self-hosted AI coding assistant. Own server + VS Code/JetBrains extensions, server-side repo indexing. Supports air-gapped via Docker export/import. Worth studying.
Cody (Sourcegraph)	Strong codebase search and context. Requires Sourcegraph server — heavier infrastructure than our approach.
Copilot / Claude Code	Cloud-only. Not applicable for air-gapped environments.
Continue.dev	Our choice. Open source, Ollama-native, custom HTTP context providers, single config for VS Code + JetBrains.

11.2 Key Design Decisions

- ChromaDB over Qdrant — file-based, no server process, simpler ops for small team
- tree-sitter over sliding window only — keeps functions/classes intact, much better retrieval quality
- Two Ollama instances over one — required to avoid serial GPU queue conflicts between chat and embed
- Single /context endpoint over two HTTP providers — works around confirmed Continue YAML bug
- DEFAULT_REPO env var over dynamic path detection — Continue sends workspace root, not repo subfolder

12. Notes & Lessons Learned

- Continue sends query in fullInput, not query field — query is always empty
- Continue workspacePath comes URL-encoded: file:///z%3A/path — must decode
- Multiple HTTP context providers in Continue YAML is a confirmed bug — only first shows up
- CUDA 13 dropped Pascal support — Tesla P4 requires CUDA 12.x
- Two Ollama instances required — single instance causes empty embeddings when chat model is busy
- ChromaDB collections get polluted if same repo indexed from different paths — always use same absolute path
- Sphinx docs / generated configs pollute RAG results — add docs/ to IGNORE_DIRS
- OLLAMA_KEEP_ALIVE env var only helps session-level — does not fix GPU queue contention
- Continue is already like a file based MPC server, it has access to tools such as read_file and those tools may need to be recreated if a custom CLI project was to be done.
- When making a custom CLI, it is important to not pigeon hole the model. For instance you may want to give the model tools to use and examples to use them, but the wording of when to use the tools is important. This needs to be looked into more. Example is if the wording is”use this tool whenever the user is trying to summarize” and the tool is summarize_repo. The tool usage could become misinterpreted and be used when trying to summarize a simple file.

13. Glossary

- RAG (Retrieval-Augmented Generation) — technique of injecting relevant documents into the model prompt before generating a response
- ChromaDB — local vector database, stores embeddings and metadata, queried by similarity
- Embedding — a vector representation of text. Similar text produces similar vectors, enabling semantic search
- tree-sitter — parser library that understands code structure in 40+ languages
- nomic-embed-text — small (274MB) embedding model, converts text to vectors for ChromaDB
- Ollama — local model runner. Wraps llama.cpp, handles model loading/unloading, exposes OpenAI-compatible API
- Continue.dev — VS Code/JetBrains extension that adds AI chat to the IDE. Connects to any Ollama endpoint
- context window — maximum number of tokens a model can process in one request. 8192 tokens for llama3.1:8b
- VRAM — GPU memory. Models must fit in VRAM for GPU acceleration. If overflow to RAM, speed drops dramatically
- tok/sec — tokens per second. Measure of inference speed. 40+ tok/sec feels real-time in an IDE